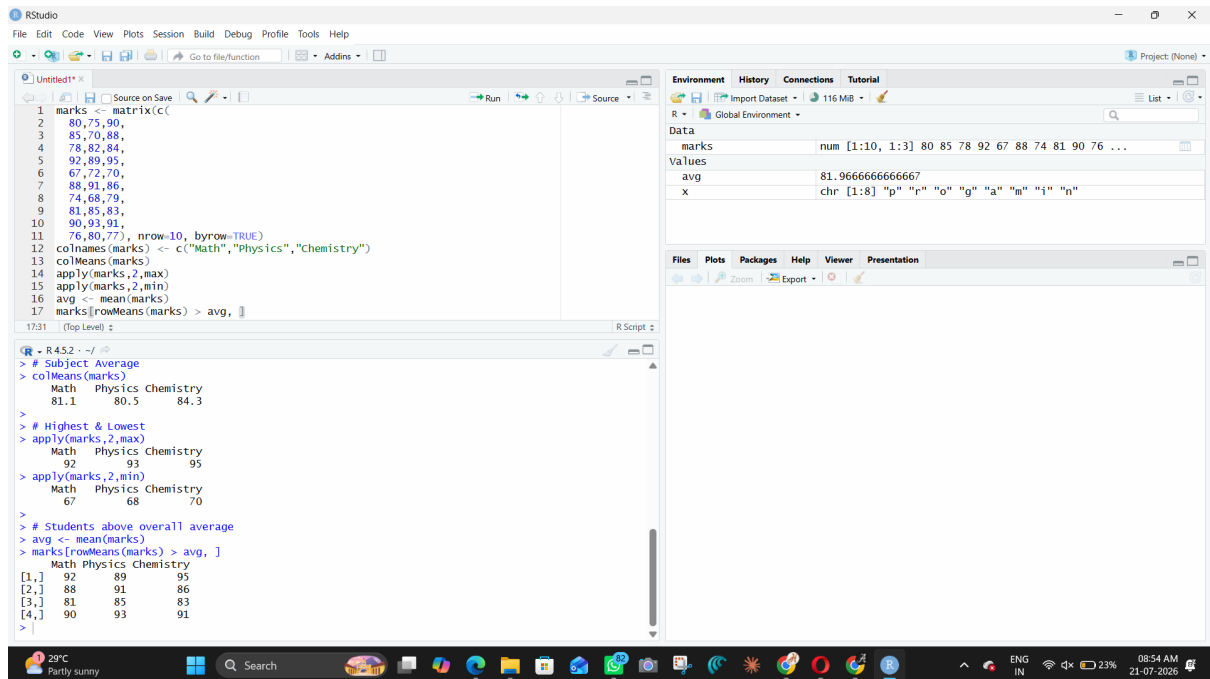


ITA0406 – R PROGRAMMING FOR STATISTICS

1)



The screenshot shows the RStudio interface. The script editor contains the following R code:

```
1 marks <- matrix(c(
2   80,75,90,
3   85,70,88,
4   78,82,84,
5   92,89,95,
6   67,72,70,
7   88,91,86,
8   74,68,79,
9   81,85,83,
10  90,93,91,
11  76,80,77), nrow=10, byrow=TRUE)
12 colnames(marks) <- c("Math","Physics","Chemistry")
13 colMeans(marks)
14 apply(marks,2,max)
15 apply(marks,2,min)
16 avg <- mean(marks)
17 marks[rowMeans(marks) > avg, ]
```

The console shows the output of the commands:

```
> # Subject Average
> colMeans(marks)
  Math Physics Chemistry
81.1    80.5    84.3

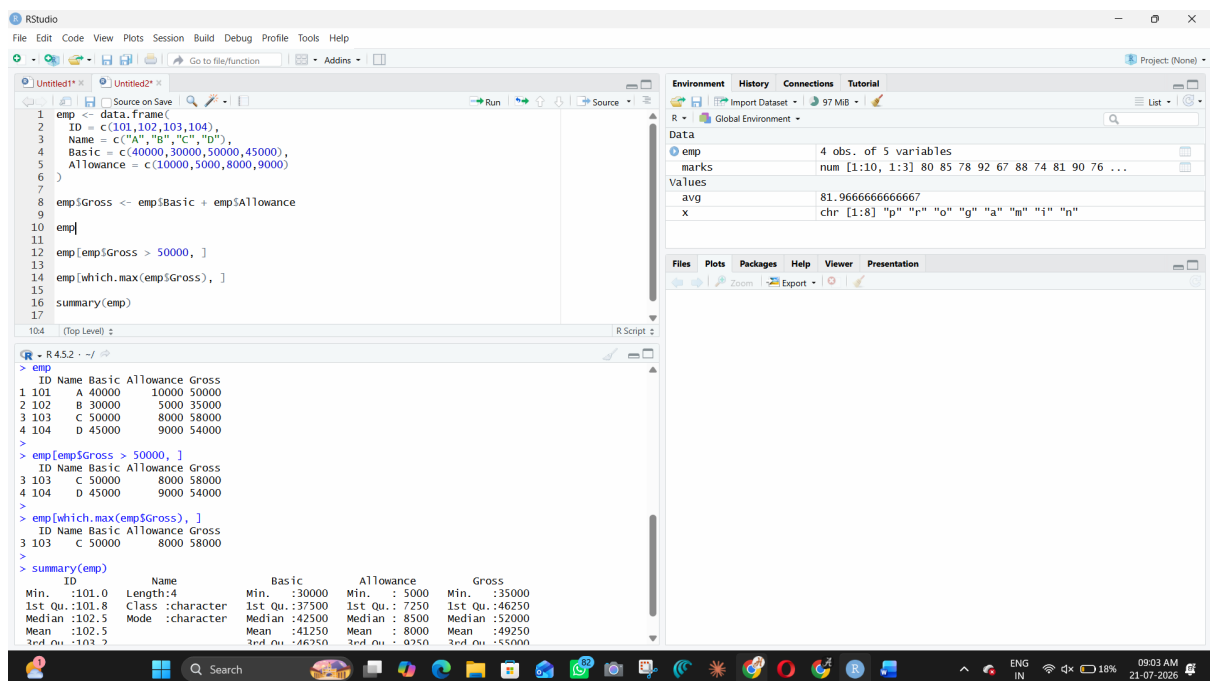
> # Highest & Lowest
> apply(marks,2,max)
  Math Physics Chemistry
  92     93     95
> apply(marks,2,min)
  Math Physics Chemistry
  67     68     70

> # Students above overall average
> avg <- mean(marks)
> marks[rowMeans(marks) > avg, ]
  Math Physics Chemistry
[1,]  92     89     95
[2,]  88     91     86
[3,]  81     85     83
[4,]  90     93     91
```

The Environment pane shows the following objects:

Object	Class	Attributes
marks	num	[1:10, 1:3] 80 85 78 92 67 88 74 81 90 76 ...
avg	dbl	81.9666666666667
x	chr	[1:8] "p" "r" "o" "g" "a" "i" "n"

2)



The screenshot shows the RStudio interface. The script editor contains the following R code:

```
1 emp <- data.frame(
2   ID = c(101,102,103,104),
3   Name = c("A","B","C","D"),
4   Basic = c(40000,30000,50000,45000),
5   Allowance = c(10000,5000,8000,9000)
6 )
7
8 emp$Gross <- emp$Basic + emp$Allowance
9
10 emp
11
12 emp[emp$Gross > 50000, ]
13
14 emp[which.max(emp$Gross), ]
15
16 summary(emp)
17
```

The console shows the output of the commands:

```
> emp
  ID Name Basic Allowance Gross
1 101  A 40000   10000 50000
2 102  B 30000    5000 35000
3 103  C 50000    8000 58000
4 104  D 45000    9000 54000

> emp[emp$Gross > 50000, ]
  ID Name Basic Allowance Gross
3 103  C 50000    8000 58000
4 104  D 45000    9000 54000

> emp[which.max(emp$Gross), ]
  ID Name Basic Allowance Gross
3 103  C 50000    8000 58000

> summary(emp)
  ID      Name      Basic      Allowance      Gross
Min.   :101.0   Length:4      Min.   :30000   Min.   : 5000   Min.   :35000
1st Qu.:101.8   Class:character 1st Qu.:37500   1st Qu.: 7250   1st Qu.:46250
Median :102.5   Mode :character   Median :42500   Median : 8500   Median :52000
Mean    :102.5                      Mean    :41250   Mean    : 8000   Mean    :49250
3rd Qu.:103.2                      3rd Qu.:46250   3rd Qu.: 9750   3rd Qu.:55000
```

The Environment pane shows the following objects:

Object	Class	Attributes
emp	data.frame	4 obs. of 5 variables
marks	num	[1:10, 1:3] 80 85 78 92 67 88 74 81 90 76 ...
avg	dbl	81.9666666666667
x	chr	[1:8] "p" "r" "o" "g" "a" "i" "n"

3)

The RStudio interface shows a script with the following code:

```

5  28,35,31,
6  30,36,33,
7  29,34,32,
8  31,35,34
9  ), dim = c(7,3),
10 dimnames = list(
11   Day = paste("Day",1:7),
12   City = c("Chennai","Delhi","Mumbai")
13 )
14
15 temp
16
17 colMeans(temp)
18
19 which(temp == max(temp), arr.ind = TRUE)
20
21 apply(temp, 2, range)

```

The console output shows the following results:

```

> temp
      City
Day 1 Chennai Delhi Mumbai
Day 2      30      34      33
Day 3      32      32      29
Day 4      31      28      34
Day 5      29      35      32
Day 6      33      31      31
Day 7      30      30      35
Day 8      31      36      34

> colMeans(temp)
Chennai Delhi Mumbai
30.85714 32.28571 32.57143

> which(temp == max(temp), arr.ind = TRUE)
      Day City
Day 7      7  2

> apply(temp, 2, range)
      City
Chennai Delhi Mumbai
[1,]      29      28      29
[2,]      33      36      35

```

The Environment pane shows the following objects:

- `emp`: 4 obs. of 5 variables
- `marks`: num [1:10, 1:3] 80 85 78 92 67 88 74 81 90 76 ...
- `temp`: num [1:7, 1:3] 30 32 31 29 33 30 31 34 32 28 ...
- `avg`: 81.9666666666667
- `x`: chr [1:8] "p" "r" "o" "g" "a" "i" "n"

4)

The RStudio interface shows a script with the following code:

```

1  product <- list(
2    ID = c(101,102,103,104),
3    Name = c("Pen","Book","Bag","Bottle"),
4    Qty = c(20,5,15,3),
5    Price = c(10,50,500,100)
6  )
7
8  value <- product$Qty * product$Price
9
10 value
11
12 product$Name[product$Qty < 10]
13
14 summary(value)

```

The console output shows the following results:

```

> apply(temp, 2, range)
      City
Chennai Delhi Mumbai
[1,]      29      28      29
[2,]      33      36      35

> product <- list(
+   ID = c(101,102,103,104),
+   Name = c("Pen","Book","Bag","Bottle"),
+   Qty = c(20,5,15,3),
+   Price = c(10,50,500,100)
+ )

> value <- product$Qty * product$Price
> value
[1] 200 250 7500 300

> product$Name[product$Qty < 10]
[1] "Book" "Bottle"

> summary(value)
      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
200.0   237.5   275.0  2062.5  2100.0  7500.0

```

The Environment pane shows the following objects:

- `emp`: 4 obs. of 5 variables
- `marks`: num [1:10, 1:3] 80 85 78 92 67 88 74 81 90 76 ...
- `product`: List of 4
- `temp`: num [1:7, 1:3] 30 32 31 29 33 30 31 34 32 28 ...
- `avg`: 81.9666666666667
- `value`: num [1:4] 200 250 7500 300

5)

The screenshot shows the RStudio interface with a script editor on the left and the Environment pane on the right. The script defines a data frame named 'patient' with columns 'Age', 'Gender', and 'Cost'. The 'Age' column has values 25, 65, 40, and 72. The 'Gender' column has values 'M', 'F', 'F', and 'M'. The 'Cost' column has values 20000, 50000, 30000, and 45000. The script then calculates the mean of the 'Cost' column and filters the data for patients aged 60 and above.

```

1 patient <- data.frame(
2   Age = c(25, 65, 40, 72),
3   Gender = c("M", "F", "F", "M"),
4   Cost = c(20000, 50000, 30000, 45000)
5 )
6
7 mean(patient$Cost)
8
9 patient[patient$Age > 60, ]
10
11 summary(patient)

```

The Environment pane shows the 'patient' data frame with 4 observations and 3 variables. The 'product' variable is a list of 4. The 'temp' variable is a numeric vector of length 12. The 'value' variable is a character vector of length 12.

The console output shows the execution of the script, including the mean of the 'Cost' column (36250) and the summary of the 'patient' data frame.

```

(R) - R 4.5.2 - ~/
+ Age = c(25, 65, 40, 72),
+ Gender = c("M", "F", "F", "M"),
+ Cost = c(20000, 50000, 30000, 45000)
+ )
> mean(patient$Cost)
[1] 36250
> patient[patient$Age > 60, ]
  Age Gender  Cost
2  65      F 50000
4  72      M 45000
> summary(patient)
      Age      Gender      Cost
Min.   :25.00   Length:4   Min.   :20000
1st Qu.:36.25   Class :character 1st Qu.:27500
Median :52.50   Mode  :character Median :37500
Mean   :50.50                      Mean   :36250
3rd Qu.:66.75                      3rd Qu.:46250
Max.   :72.00                      Max.   :50000

```

6)

The screenshot shows the RStudio interface with a script editor on the left and the Environment pane on the right. The script defines two functions: 'checkBalance' and 'transactionHistory'. The 'checkBalance' function prints the current balance, and the 'transactionHistory' function prints the transaction history. The script then simulates a banking transaction by depositing 500 and withdrawing 300, followed by checking the balance and the transaction history.

```

17 }
18
19 checkBalance <- function() {
20   print(balance)
21 }
22
23 transactionHistory <- function() {
24   print(history)
25 }
26
27 deposit(500)
28
29 withdraw(300)
30
31 checkBalance()
32
33 transactionHistory()

```

The Environment pane shows the 'history' variable as a character vector of length 12, containing the transaction history. The 'value' variable is a numeric vector of length 12, containing the transaction values. The 'x' variable is a character vector of length 12, containing the transaction descriptions.

The console output shows the execution of the script, including the function definitions and the simulation results.

```

(R) - R 4.5.2 - ~/
+   print(balance)
+ }
> checkBalance <- function() {
+   print(balance)
+ }
> transactionHistory <- function() {
+   print(history)
+ }
> deposit(500)
> withdraw(300)
> checkBalance()
[1] 1200
> transactionHistory()
[1] "Deposited 500" "Withdrawn 300"

```

7)

The screenshot shows the RStudio interface. The script pane contains the following code:

```

1 name <- c("A", "B", "C", "D", "E")
2
3 age <- c(17, 20, 15, 25, 18)
4
5 eligible <- age >= 18
6
7 data.frame(Name = name, Age = age, Eligible = eligible)
8
9 sum(eligible)
10
11 sum(!eligible)
12
13 (sum(eligible) / length(age)) * 100

```

The console shows the execution results:

```

> age <- c(17, 20, 15, 25, 18)
> eligible <- age >= 18
> data.frame(Name = name, Age = age, Eligible = eligible)
  Name Age Eligible
1   A  17    FALSE
2   B  20     TRUE
3   C  15    FALSE
4   D  25     TRUE
5   E  18     TRUE
> sum(eligible)
[1] 3
> sum(!eligible)
[1] 2
> (sum(eligible) / length(age)) * 100
[1] 60

```

The environment pane shows the following objects:

name	value
name	chr [1:5] "A" "B" "C" "D" "E"
age	num [1:4] 200 250 7500 300
x	chr [1:8] "p" "r" "o" "g" "a" "m" "i" "n"

The Functions pane shows the following functions:

Function	Definition
checkBalance	function ()
deposit	function (amount)
transactionHistory	function ()
withdraw	function (amount)

8)

The screenshot shows the RStudio interface. The script pane contains the following code:

```

3 90,110,100,
4 140,150,145,
5 80,95,100
6 ), nrow = 4, byrow = TRUE)
7
8 rownames(sales) <- c("Product1", "Product2", "Product3", "Product4")
9 colnames(sales) <- c("Region1", "Region2", "Region3")
10
11 sales
12
13 colSums(sales)
14
15 rowSums(sales)
16
17 rownames(sales)[which.max(rowSums(sales))]
18
19 colnames(sales)[which.max(colSums(sales))]

```

The console shows the execution results:

```

> sales
  Region1 Region2 Region3
Product1  100   120   130
Product2   90   110   100
Product3  140   150   145
Product4   80    95   100
> colSums(sales)
  Region1 Region2 Region3
    410    475    475
> rowSums(sales)
  Product1 Product2 Product3 Product4
    350    300    435    275
> rownames(sales)[which.max(rowSums(sales))]
[1] "Product3"
> colnames(sales)[which.max(colSums(sales))]
[1] "Region2"

```

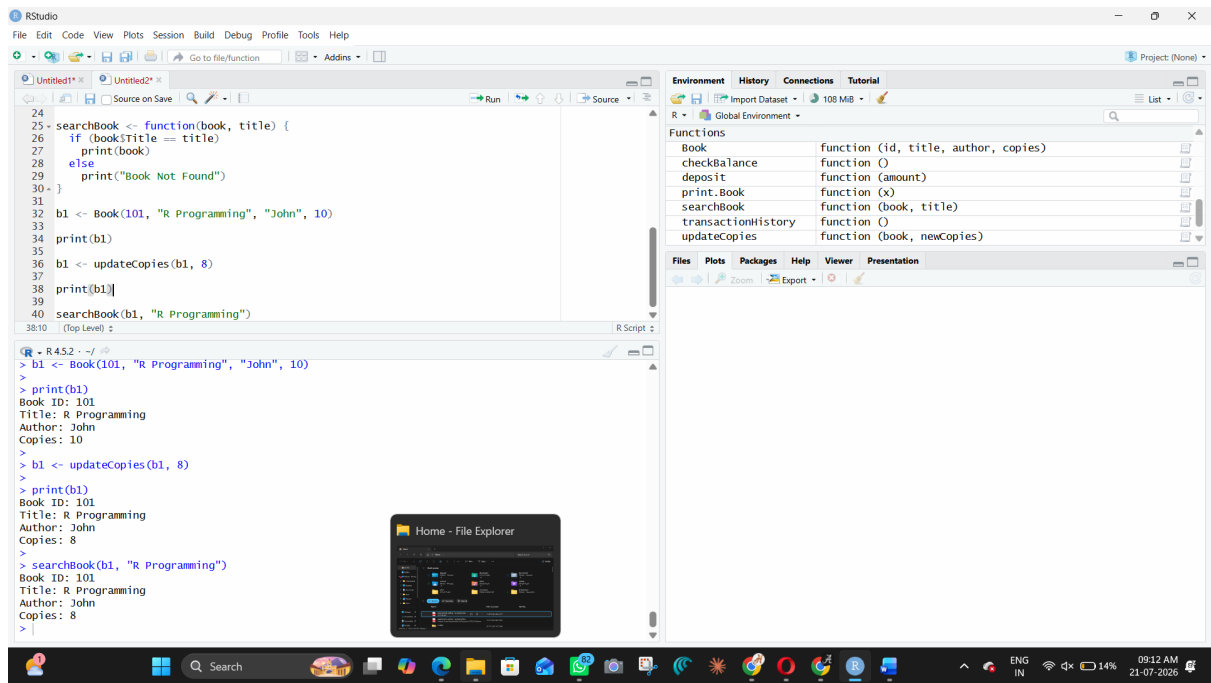
The environment pane shows the following objects:

name	value
sales	num [1:4, 1:3] 100 90 140 80 120 110 150 95 130 100 ...
temp	num [1:7, 1:3] 30 32 31 29 33 30 31 34 32 28 ...

The Values pane shows the following values:

name	value
age	num [1:5] 17 20 15 25 18
avg	81.9666666666667
balance	1200
eligible	logi [1:5] FALSE TRUE FALSE TRUE TRUE
history	chr [1:2] "Deposited 500" "Withdrawn 300"

9)



10)

